# Variant del Recorregut d'un Tauler d'Escacs: Resolució del Camí Hamiltonià Bipartit amb Restricció de No-Captura

Josep Oliver Vallespir, Hugo Valls Sabater

**Resum**—Aquest document detalla l'arquitectura, el disseny orientat a objectes i l'optimització d'una aplicació desenvolupada en Java, la finalitat de la qual és resoldre una variant amb dues peces del problema del Camí Hamiltonià sobre un tauler d'escacs. En aquesta versió, dues peces, iguals o diferents, alternen torns per visitar la totalitat de les caselles del tauler d'escacs sense que, en cap moment de la iteració, es vulneri la restricció crítica de no-captura (cap peça pot situar-se sota la línia d'amenaça de l'altra). A través d'aquest text, s'exposarà de manera completa l'aplicació del patró Model-Vista-Controlador (MVC), la gestió de la concurrència mitjançant fils d'execució independents, la implementació del polimorfisme per establir l'herència de les peces, i les podes intel · ligents aplicades a l'arbre de recursivitat del Backtracking per domar i controlar la complexitat exponencial del problema.

**Index Terms**—Backtracking, Camí Hamiltonià, Concurrència, Java Swing, Patró MVC, Polimorfisme, Thread-Safety, Volatile.

## 1 Introducció

L'estudi dels recorreguts en grafs és una part fonamental dins de les ciències de la computació i les matemàtiques discretes. El problema clàssic del recorregut del cavall ens planteja trobar una seqüència de moviments viables de manera que la peça passi per cadascun dels nodes del graf, en el nostre cas el tauler d'escacs, exactament una vegada, generant així un *Camí Hamiltonià* [**?**].

Amb això, a n'aquesta pràctica elevam una mica aquesta complexitat introduïnt com una espècie de multijugador on operen dues peces a la vegada. La condició d'èxit de l'algorisme requereix no només cobrir l'àrea del tauler $N \times N$, sinó fer-ho complint amb una restricció de seguretat mútua: la regla de no-captura. Conseqüentment, cada node de decisió dins de l'espai de cerca no només ha de validar els límits del tauler i la disponibilitat de la casella, sinó que ha de calcular els vectors d'atac de les dues peces per descartar col·lisions imminents.

Amb aquest document, esmentarem com s'ha estructurat el nostre programa per satisfer aquestes necessitats y explicarem les decisions de codi que hem implementat, mantenint l'escalibilitat de la interfície gràfica, i la reusabilitat de les dades.

## 2 Estructura del Programa

Amb l'objectiu de complir amb els patrons de disseny habituals de les interfícies d'escriptori actuals, l'arquitectura de l'aplicació està estructurada per el patró de disseny del Model-Vista-Controlador teoritzat a classe de Algorismes Avançats [1].

A continuació mostram la estructura de classes i packages en la que hem definit el nostre programa:

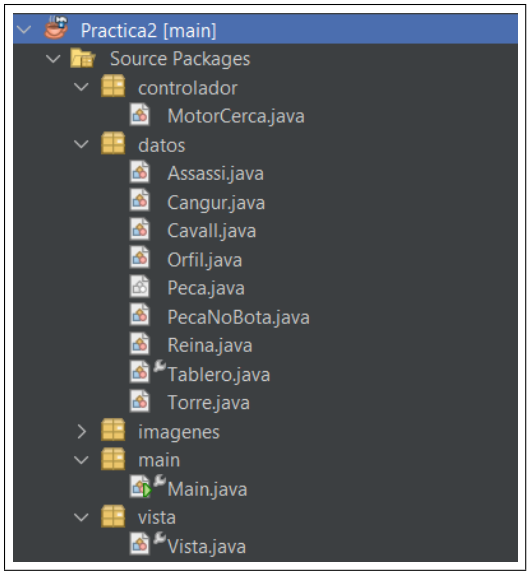


Figura 1. Representació de la estructura del projecte.

Seguidament, procedirem a explicar component per component tot el programa:

## 3 La Vista: Interfície d'Usuari (GUI)

A l'hora de dissenyar la interfície gràfica amb Java Swing, el nostre objectiu principal ha estat crear un entorn net, intuïtiu i, sobretot, completament separat

- *Estudiants del Grau en Enginyeria Informàtica a la Universitat de les Illes Balears (UIB).*
  *Assignatura: Algorismes Avançats.*

del motor de cerca de l'aplicació (Controlador). Hem fet servir un gestor d'espais `BorderLayout`, hem dividit la pantalla principal en tres grans zones ben diferenciades.

### 3.1 Panell Lateral: Configuració del Joc

La part esquerra de la pantalla (`BorderLayout.WEST`) funciona com el nostre tauler de control principal abans de començar a jugar. L'hem organitzat verticalment amb un `BoxLayout` perquè tots els menús i botons quedin ben apilats i ordenats.

Aquí l'usuari troba els botons d'"Iniciar Partida" i "Aturar". Un detall important de disseny és que, quan es prem Iniciar, el botó es desactiva automàticament. Això evita que l'usuari l'apreti diverses vegades per error, cosa que crearia fils d'execució duplicats i podria penjar el programa. Des d'aquí també es pot triar la mida del tauler amb un selector numèric (`JSpinner`) que actualitza la quadrícula en temps real.

Pel que fa a la selecció de peces per a cada jugador, hem volgut fer-ho molt visual i intuïtiu. Hem posat una graella amb les icones de les sis peces disponibles. Quan l'usuari assigna una peça al Jugador 1, el botó d'aquesta peça s'emmarca amb una línia de color verd; si l'assigna al Jugador 2, s'emmarca de color blau. El detall més interessant és que, si tots dos jugadors trien exactament la mateixa peça (per exemple, dos Cavalls), es pinten dues vores, una en verd i l'altra en blau. D'aquesta manera, l'usuari sap ràpidament com està configurada la partida sense haver de llegir cap text explicatiu.

### 3.2 Panell Inferior: Informació i Retard

A la part inferior de la finestra (`BorderLayout.SOUTH`) hi tenim la barra d'estat i el control de velocitat.

La barra d'estat és un text que s'actualitza constantment per dir-nos què està fent l'algorisme en aquell moment. Canvia de color per fer-se més evident: es posa blava quan està cercant una solució, verda si en troba una (indicant el temps que ha trigat), i vermella si determina que el problema no té solució possible. Cal dir, que sempre és mostra el temps, tant si aturam l'execució, com si troba o no la solució.

Al seu costat hi ha un element important: el control de velocitat (`JSlider`), que permet pausar l'animació de les peces entre 0 i 100 mil·lisegons. Aquest control no només serveix per fer l'animació més ràpida o més lenta. Per defecte, la barra es troba al mínim (0 ms), i li diem a l'algorisme que deixi d'enviar ordres de redibuixar la pantalla pas a pas. Com que el processador ja no perd temps dibuixant gràfics fotograma a fotograma, pot dedicar tota la seva potència a calcular la matemàtica del Backtracking, resolent taulers complexos de forma gairebé instantània. En el cas en que l'usuari posi un retard, s'anirà repintant el panell pas a pas, en funció de l'algorisme de backtracking, com més retard, més tardarà a mostrar la solució, si existeix.

### 3.3 Panell Central: el Tauler

Finalment, al centre de la pantalla (`BorderLayout.CENTER`) hi tenim el tauler d'escacs. En lloc de crear una matriu enorme plena de `JButtons` per fer la graella (cosa que consumiria molta memòria i faria que l'aplicació anés lenta), hem decidit dibuixar el tauler nosaltres mateixos manualment.

Per fer-ho, hem sobreescrit el mètode de Java `paintComponent(Graphics g)`. Això té una gran avantatge pràctica: a cada instant, el programa llegeix quina mida té la finestra de l'ordinador. Si l'usuari la fa més gran o més petita estirant-la amb el ratolí, el codi recalcula les mides de les caselles i el tauler s'adapta fluidament a la nova pantalla.

Cal dir que aquesta classe Tauler també forma part del model de dades ja que emmagatzema la dimensió i la matriu del tauler, la qual cosa ens és indispensable per aplicar l'algorisme.

Per pintar el tauler, el programa llegeix la matriu lògica de dades i va dibuixant tres capes una a sobre de l'altra:

1) **El fons:** Pinta les caselles alternant colors clars i foscos perquè sembli un tauler d'escacs real.
2) **Les peces:** Si la matriu de nombres indica que a una casella hi ha un pas registrat, el programa mira si és un pas parell o imparell i dibuixa la imatge del Jugador corresponent ben centrada.
3) **Els números:** Per últim, escriu el número del pas a la casella de color verd. Això deixa un rastre visual clar perquè l'usuari pugui seguir quin camí exacte ha resolt el nostre algorisme pas a pas.

A continuació es pot observar com està estructurada la nostra interfície al executar el programa:

Figura 2. Interfície gràfica inicial.

## 4 El Model: Disseny i Estructura del Model de Dades

Dins de l'arquitectura MVC que hem implementat, el Model de Dades representa el "cervell" pur de l'aplicació. La nostra premissa de disseny ha estat molt clara: cap classe d'aquest paquet pot saber res sobre colors, píxels o botons. Tot el que hi ha aquí és matemàtica i estat lògic, dissenyat específicament per alimentar l'algorisme de Backtracking de la forma més ràpida i eficient possible.

### 4.1 El Tauler (`Tablero.java`)

Una de les decisions més importants que hem pres ha estat tractar el Tauler no com un simple element visual, sinó com l'estructura de dades central de la nostra aplicació (ubicant-lo al paquet de dades). En lloc de passar matrius per tot el codi, hem encapsulat l'estat del joc dins de la classe `Tablero`.

Aquesta classe emmagatzema la matriu bidimensional de sencers ($N \times N$) que fa de mapa. Actua com a base per a l'algorisme: s'encarrega d'emmagatzemar quines caselles estan lliures, quines estan ocupades per un pas concret, i quines són les dimensions màximes d'aquest tauler. Centralitzar això en una classe pròpia ens ha permès crear mètodes auxiliars que validen si una coordenada està dedins o defora dels límits (OutOfBounds). D'aquesta manera, llevam pes a l'algorisme principal, que només ha de demanar al Tauler si un moviment és legal o no, mantenint el codi molt més estructurat i segur.

### 4.2 Les Peces

Per modelar les peces del joc, hem evitat fer codi espagueti ple de condicionals (com ara `if (peca == "Cavall") else ...`). En es seu lloc, hem utilitzat la Programació Orientada a Objectes creant una jerarquia de classes.

Com a classe Pare tenim la classe abstracta `Peca`. Aquesta classe defineix el que serà una peça del tauler d'escacs, o sigui, obliga a qualsevol peça que vulguem crear a saber calcular els seus propis moviments possibles i a saber calcular quines caselles ataca. Gràcies a això, el motor de cerca pot agafar qualsevol peça i demanar-li on pot anar, sense importar-li gens ni mica si és un Cavall, una Reina o un Cangur. Això és el que es coneix com a polimorfisme, i ens garanteix que si demà volem afegir 10 peces noves al joc, no haurem de tocar res de l'algorisme de Backtracking.

A més, durant la realització de la pràctica, ens vàrem adonar de que la Torre, l'Òrfil i la Reina comparteixen un comportament idèntic: totes tres "patinen" en línia recta fins que xoquen contra un obstacle/vora. Per no repetir el mateix bucle a les tres classes, hem creat una classe intermèdia anomenada `PecaNoBota` (que hereta de `Peca`). Hem programat la lògica de "patinatge" dins aquesta classe, i hem fet que la Torre, l'Òrfil i la Reina heretin.

Per aconseguir que cada peça sàpiga com es mou sense complicar-nos la vida, hem reduït els moviments d'escacs a vectors de l'estil ($\Delta x, \Delta y$). A l'interior de cada classe específica (com `Cavall` o `Assassi`), hi hem declarat una petita matriu privada amb les seves direccions. Per exemple, els 8 salts en "L" del Cavall o els vectors rectes de la Torre. Quan el Tauler o l'algorisme li demana a la peça on pot anar, aquesta simplement suma els seus vectors a la seva coordenada actual i retorna una llista de resultats vàlids.

Finalment, per connectar els botons de la interfície gràfica amb aquesta estructura de dades, hem implementat un patró de disseny anomenat *Factory Method* [2] dins de la classe base. Quan l'usuari tria "Reina" a la pantalla i prem Iniciar, la interfície li passa aquesta paraula a la Factoria. Aquest mètode s'encarrega de traduir la paraula, construir l'objecte `Reina` corresponent, assignar-li si és el Jugador 1 o el Jugador 2, i enviar-lo per aplicar a l'algorisme. És frontera neta i perfecta entre allò que l'usuari veu i les matemàtiques que resolen el problema.

A continuació, es detalla la implementació de cada peça i el seu impacte sobre l'arbre de cerca:

Les peces tradicionals s'han modelat respectant la seva naturalesa geomètrica, dividint-se en peces que boten i peces que patinen.

- **El Cavall:** Representa una peça que bota certes caselles. Funciona mitjançant un conjunt de vuit vectors (en forma d'"L", com ara $\Delta x = \pm 1, \Delta y = \pm 2$). La seva lògica interna ignora totes les caselles intermèdies. I només realitza comprovacions de límits (*OutOfBounds*) i d'ocupació de la casella destí. Aquest comportament li garanteix una alta taxa de viabilitat i supervivència en fases avançades del *Backtracking*, ja que no queda atrapat per parets de caselles prèviament visitades.
- **La Torre:** És l'exemple perfecte de peça que "patina" en lloc de botar. Es mou exclusivament en línia recta (vertical o horitzontal). A nivell de codi, la Torre avança casella a casella utilitzant un bucle `while`. A cada petita passa que fa, li demana a la matriu del Tauler si aquella casella ja ha estat trepitjada prèviament (és a dir, si té un número major que zero). Si és així, s'atura i no pot continuar avançant per aquella línia. Aquest comportament provoca que, a mesura que la partida avança i el tauler s'omple de moviments, la Torre es queda ràpidament sense espai, atrapada darrere de les parets de caselles ja visitades. Per aquest motiu, és una de les peces més difícils de fer quadrar per aconseguir completar tot el tauler sense que es quedi bloquejada.

- **L'Òrfil (Alfil):** Funciona de manera molt semblant a la Torre (també "patina" fins a trobar un obstacle), però movent-se exclusivament en diagonal. Això fa que tingui una limitació molt forta: sempre es mourà per caselles del mateix color. A nivell pràctic, això significa que l'Òrfil només té accés a la meitat del tauler. Aquesta restricció matemàtica fa que sigui totalment impossible resoldre el problema si l'altra peça no és capaç de cobrir l'altra meitat de les caselles (les del color contrari).
- **La Reina:** És la peça més versàtil perquè combina els moviments de la Torre i de l'Òrfil alhora, poguent avançar en vuit direccions diferents. Com que té tantes opcions de moviment des de qualsevol casella, genera un arbre de possibilitats enorme que l'algorisme ha d'explorar. No obstant això, a la nostra pràctica, com que té un radi d'acció tan gran, amenaça moltíssimes caselles alhora i romp contínuament la "Regla de No-Captura". L'algorisme es passa gran part del temps descartant moviments per evitar col·lisions, cosa que fa que intentar resoldre un tauler on hi hagi una Reina requereixi molta més potència i temps de càlcul.

Apart de les peces originals del joc d'escacs, hem inventat dues peces completament noves. Aquestes peces obliguen al *Backtracking* a enfrontar-se a formes de moure's i d'atacar diferents.

- **El Cangur:** És una peça pensada per fer bots, tots iguals. A nivell de codi, li hem assignat uns vectors que l'obliguen a botar sempre exactament 2 caselles cap a qualsevol direcció (tant en línia recta com en diagonal). Com que fa salts llargs, té una gran facilitat per esquivar bloquejos i caselles ja trepitjades. No obstant això, té un defecte gravíssim de cara a l'algorisme: com que mai es pot moure només 1 casella per posar-se a la del costat, quan el tauler està gairebé ple (cap al final de la partida) acostuma a deixar "forats" o caselles aïllades on no hi pot caure. Això fa que l'algorisme perdi moltíssim temps desfent passos anteriors en adonar-se que no podrà completar el tauler.

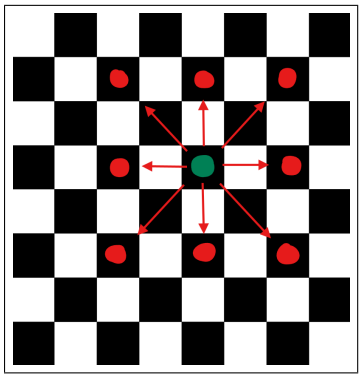

Figura 3. Possibles Moviments del Cangur a un tauler de 8x8.

- **L'Assassí:** Aquesta peça l'hem dissenyat expressament per dur l'algorisme a un altre nivell. És una peça "híbrida" perquè té dues maneres de moure's totalment diferents. D'una banda, es pot moure només 1 casella en horitzontal o vertical, cosa que li va perfecte per moure's a poc a poc per espais tancats. Però, d'altra banda, pot fer salts molt grans d'exactament 3 caselles en diagonal. Quan l'algorisme analitza on pot anar o atacar aquesta peça, es troba amb un arbre de decisions una mica estrany: en el mateix torn ha de calcular atacs a la casella del costat i atacs gairebé a l'altra punta del tauler. Si el programa és capaç de gestionar aquesta peça sense penjar-se i evitant col·lisions, ens demostra a la pràctica que el nostre sistema de poda és 100% flexible i robust.

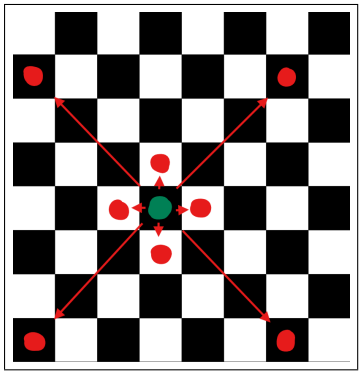

Figura 4. Possibles Moviments de l'Assassí a un tauler de 8x8.

## 5 El Controlador: Algorisme de Backtracking

El nucli de la nostra pràctica resideix a la classe `MotorCerca.java`. Aquest mòdul actua com a Controlador dins del patró MVC: és l'encarregat d'agafar el Tauler i les Peces (el Model), fer-los interactuar seguint les regles programades, i dir-li a la pantalla (la Vista) quan s'ha d'actualitzar.

### 5.1 Fils d'Execució i el bloqueig de la pantalla

El primer gran problema que ens vam trobar en programar la cerca recursiva va ser que, en clicar "Iniciar", la finestra del programa es quedava completament penjada fins que trobava la solució. Això passa perquè Java Swing utilitza un únic fil per dibuixar la pantalla i detectar els clics. Si posem aquest fil a calcular milions de moviments, la interfície mor.

Per solucionar-ho, vam fer que la classe `MotorCerca` hereti de la classe `Thread`. D'aquesta manera, quan l'usuari prem el botó, l'algorisme s'executa en un fil secundari completament independent en un altre nucli del processador. Això permet que la pantalla continuï funcionant de forma fluida i que l'usuari pugui prémer el botó "Aturar" en qualsevol moment.

A més, per garantir que el botó "Aturar" funcioni a l'instant, hem utilitzat una variable booleana amb la directiva `volatile` (`volatile boolean aturat`). Això és crucial, ja que obliga al processador a llegir l'estat d'aquesta variable directament de la memòria principal a cada pas de l'algorisme, evitant que l'ordinador guardi un valor antic a la seva memòria cau i ignori la nostra ordre d'aturar el programa.

### 5.2 L'Algorisme de Backtracking

El mètode principal és una funció recursiva anomenada `resoldre`, que aplica una estratègia de cerca en profunditat (DFS o *Backtracking*). La seva estructura seqüencial és la següent:

1) **Condició d'Èxit:** El primer que fa la funció és mirar si el comptador de passos ha superat el total de caselles del tauler ($N \times N$). Si és així, vol dir que hem omplert tot el tauler i retorna cert (`true`), aturant la cerca.
2) **Gestió del Torn:** A continuació, l'algorisme fa una simple divisió exacta (`pas % 2 != 0`) per saber de qui és el torn. Si el pas és imparell, agafa la peça del Jugador 1; si és parell, la del Jugador 2. Llavors demana a la peça corresponent la seva llista de moviments possibles.
3) **La Poda (Regla de No-Captura):** Aquest és el punt més crític. Abans de fer qualsevol moviment de la llista, comprovam que es segur. L'algorisme utilitza els mètodes de la peça per fer-se dues preguntes vitals: *"Si me coloc aquí, atac a l'altre jugador?"* i *"Si me coloc aquí, me pos davall el radi d'atac de l'altre jugador?"*. Si alguna d'aquestes dues respostes és afirmativa, l'algorisme descarta aquella casella a l'instant. Aquesta decisió de disseny s'anomena "poda", i ens estalvia haver de calcular milions de camins inútils que acabarien trencant les regles de la pràctica.
4) **Marcar, Avançar i Desfer:** Si la casella és segura, l'algorisme hi escriu el número del pas actual. En aquest punt, es demana a la interfície gràfica que es redibuixi (sempre que el retràs de repintat no estigui activat) i la funció es crida a si mateixa per calcular el pas següent (`pas + 1`). Si aquesta crida futura acaba tornant `false` (vol dir que ens hem quedat sense sortida), l'algorisme esborra el número de la casella (hi posa un 0) i prova amb la següent opció de la llista.
   Aquesta és la clau del nostre *Backtracking*.

### 5.3 Millores en el Rendiment

Durant les primeres proves, vam veure que l'algorisme resolia taulers de $5 \times 5$ molt ràpidament, però es quedava calculant hores infinites amb un tauler normal de $8 \times 8$. Com que el nombre de camins possibles és molt elevat, si anem provant els moviments "a cegues" (per exemple, sempre de dalt a baix), l'algorisme acostuma a deixar caselles buides a les cantonades. Quan s'adona de l'error, ha de desfer gairebé tot el tauler.

Per evitar aquesta explosió exponencial de temps, vam decidir programar una adaptació de la Regla de Warnsdorff [3]. On aplica que just després d'obtenir la llista de moviments legals d'una peça, l'algorisme no els prova directament. En lloc d'això, simula cada moviment i compta *quants moviments futurs* ens quedarien lliures des d'aquella nova casella.

Una vegada comptats, ordenam la llista de moviments de menor a major. D'aquesta manera, l'algorisme prioritza sempre moure's cap a les caselles que tenen **menys** sortides (com les vores o els caires del tauler). Aquesta simple decisió força a les peces a anar colocant-se a les caselles de més difícil accés al principi, deixant el centre (on és fàcil moure's) per al final. Gràcies a aquesta heurística, un problema de $8 \times 8$ que tardaria dies en resoldre's per força bruta, el nostre codi el resol en molt menys temps, fent així una cerca molt més intel·ligent, encara que haguem d'ordenar la llista de moviments a cada recursió.

## 6 El main (`Main.java`)

Un cop explicat tota l'aplicació, procedirem a explicar com s'executa l'aplicació.

Teníem clar que no podíem simplement instanciar la finestra dins del mètode `main`. La documentació oficial de Java Swing ens adverteix que aquesta llibreria no és segura respecte als fils d'execució (*Thread-safe*) [4].

El mètode `main` de qualsevol programa en Java s'executa en el seu propi fil base (el *Main Thread*), mentre que tota la interfície gràfica i la interacció amb l'usuari funciona sobre un fil especialitzat anomenat EDT (*Event Dispatch Thread*). Si intentam construir els panells, el tauler i els botons directament des del fil

principal, ens arrisquem a patir condicions de carrera o bloquejos ocults fins i tot abans de començar a jugar.

Per evitar qualsevol error d'aquest tipus, a la nostra classe Main hem encapsulat la creació de la `Vista` dins de la instrucció `SwingUtilities.invokeLater(...)`. El que fa aquesta comanda és empaquetar l'ordre d'inici i posar-la a la cua de tasques de l'EDT. D'aquesta manera, deleguem el control absolut de l'arrencada al fil gràfic, garantint que la interfície es construeix de forma totalment sincronitzada.

Cal dir que al main tan sols instanciam la vista, ja que el model de dades es crearà quan es crei la vista i determinem la dimensió del tauler i les peces.

## 7 Casos d'Ús i Execucions Destacades

Per comprovar la robustesa del nostre codi, hem sotmès a l'aplicació a diferents escenaris. A continuació, mostrem i analitzem tres execucions clau que il·lustren perfectament com es comporta l'algorisme davant de diferents reptes..

### 7.1 Cas 1: Cavall vs. Cavall en 8x8

El primer cas de prova és un escenari molt clàssic. Com que tots dos jugadors utilitzen la mateixa peça, l'algorisme no sofreix asimetries a l'hora de calcular els moviments.

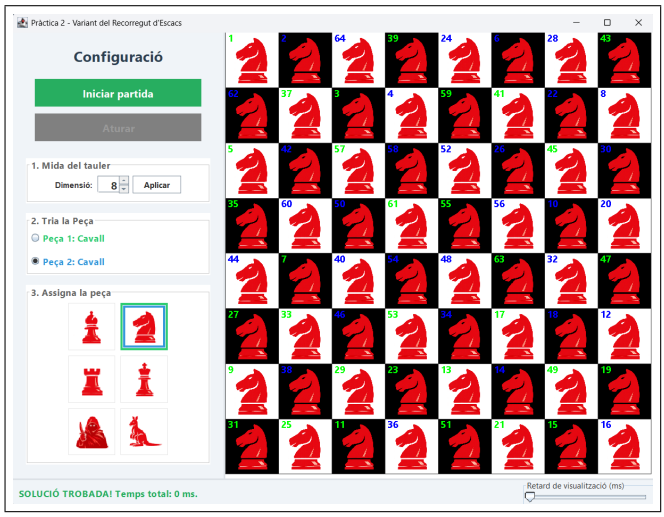


Figura 5. Tauler de 8x8 resolt exitosament per dos Cavalls.

Gràcies a la implementació de l'heurística de Warnsdorff, l'algorisme obliga als dos cavalls a anar resseguint les vores del tauler primer, deixant la part central lliure. Com es pot apreciar a la Figura 5, el rastre numèric demostra com les peces van saltant per les caselles més crítiques abans de tancar-se, va primer per les vores. Com que el Cavall bota i no es bloqueja amb les caselles ja visitades, aquest escenari es resol de forma gairebé instantània.

### 7.2 Cas 2: Torre vs. Òrfil en 5x5

Aquest és un dels casos més interessants que vam trobar durant les proves. Si posem a jugar una Torre contra un Òrfil en un tauler imparell, ens adonem que el programa s'atura en uns 500ms i ens avisa que no hi ha solució (part inferior).

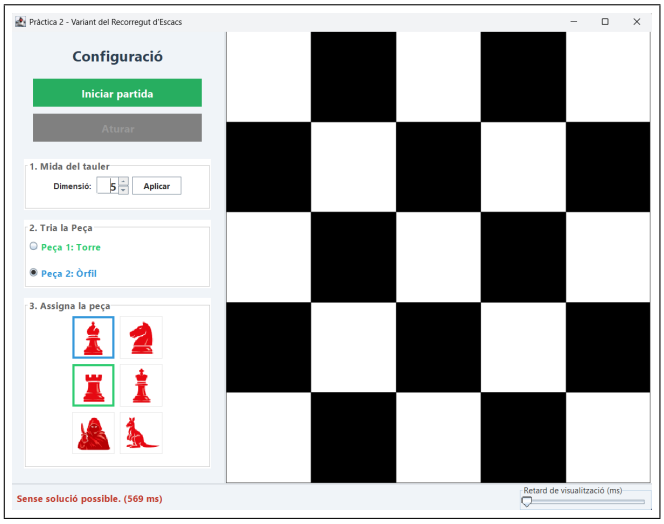


Figura 6. Tauler de 5x5 no resolt per una Torre i un Òrfil.

L'explicació neix en les limitacions de les peces. L'Òrfil (Jugador 2) es mou en diagonal, de manera que està condemnat a moure's només per les caselles d'un únic color. La Torre (Jugador 1), que es mou en línia recta, ràpidament crea un "mur" horitzontal i vertical que talla el tauler per la meitat. Com que la Torre no pot saltar les caselles que l'Òrfil ja ha trepitjat, i l'Òrfil només té accés a la meitat del tauler, el *Backtracking* s'adona de seguida que les peces s'acaben ofegant mútuament. És un èxit del nostre sistema de poda que no es quedi hores cercant on no hi ha res a fer.

### 7.3 Cas 3: Cangur vs. Assassí

Finalment, varem executar l'algorisme amb les dues peces noves que varem dissenyar en un tauler més gran de 10x10.

El Cangur fa salts de 2 caselles exactes, mentre que l'Assassí alterna passes curtes d'una casella amb salts diagonals enormes de 3. Durant l'execució d'aquesta partida amb retard visual (posant el `JSlider` a 50 ms), vam poder observar com el Cangur anava deixant caselles solitàries ("forats") al seu pas perquè és incapaç de fer passes curtes. Això va obligar l'algorisme a fer passos enrere constantment, esborrant camins que semblaven bons per culpa d'aquests forats inaccessibles. Tot i trigar molt més temps que els Cavalls clàssics, la partida demostra que el motor pot avaluar amenaces molt estranyes sense penjar-se.

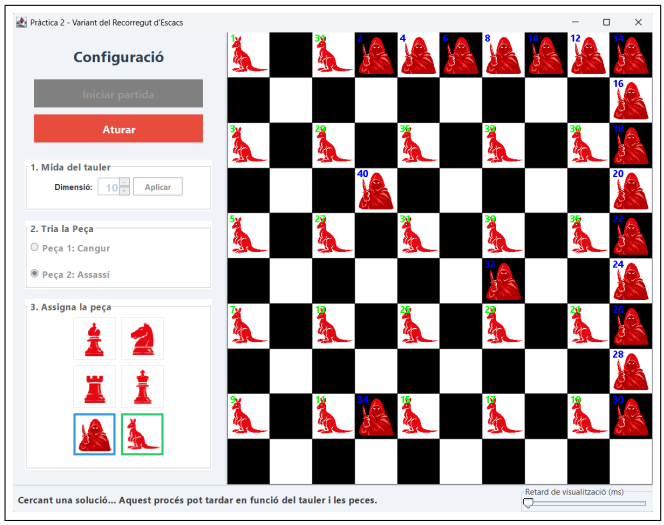


Figura 7. Tauler de 10x10 amb un Cangur i un Assassí.

## 8 Conclusió i Valoració Personal

Arribats al final d'aquesta pràctica, podem afirmar que el desenvolupament d'aquest projecte ha estat un repte molt més exigent, però visualment molt més atractiu que la primera pràctica. Al principi, semblava que l'única dificultat seria programar l'algorisme de *Backtracking*. No obstant això, a mesura que avançàvem, ens vam adonar que fer funcionar la part més matemàtica només era la meitat de la feina; l'altra meitat era aconseguir que tot el sistema aguantés el pes d'aquests càlculs sense quedar-se penjat.

A nivell algorísmic, el que hem après és que la força bruta no serveix per res quan xerram de complexitat exponencial. Veure com el nostre programa es quedava calculant durant bastant de temps per resoldre un simple tauler de 6x6 va ser un poc frustrant. Però justament per això, el moment en què vam implementar l'Heurística de Warnsdorff i vam veure com el mateix tauler es resolia en pocs mil·lisegons va ser un punt molt important de la pràctica. Entendre que ordenar les llistes de moviments (prioritzant les caselles amb menys sortides) canviava radicalment el rendiment ens ha fet veure la importància de pensar una mica més el que fa l'algorisme. També estem molt orgullosos del nostre sistema de "poda" basat en la regla de no-captura, que talla branques de l'arbre de recursivitat de forma molt eficient abans de perdre temps explorant camins inútils.

D'altra banda, hem hagut d'aprendre, si o si, la importància de separar la interfície gràfica de les dades i de la lògica. Si haguéssim mesclat el codi de la interfície amb el codi de les peces, hauria estat impossible implementar el fil d'execució independent (`Thread`). Aconseguir que l'usuari pugui prémer el botó "Aturar" en qualsevol moment i que la pantalla no es quedi congelada ens ha fet entendre de veritat com funciona la concurrència a Java (amb eines com `SwingUtilities.invokeLater` o les variables `volatile`).

En conclusió, la nostra valoració personal és bona, creiem que hem fet una bona feina. Ha estat una pràctica llarga, amb colque mal de cap a l'hora d'ajustar el repaint amb la velocitat de la cerca recursiva. Però el resultat final és una aplicació robusta, eficient i molt visual que demostra que hem comprès no només la teoria de backtracking de l'assignatura, sinó també com construir programari de millor qualitat i escalable per al món real.

## Referències

[1] Assignatura d'Algorismes Avançats, 2026.
[2] Google Gemini AI *Asistencia en Interficies y Factory Method*
[3] H. C. von Warnsdorff, *La solución más simple y general del recorrido del caballo*. Schmalkalden, 1823.
[4] Oracle Corporation, "Concurrency in Swing," *The Java Tutorials*. [En línia]. Disponible a: https://docs.oracle.com/javase/tutorial/uiswing/concurrency/index.html.